TROUBLESHOOTING

HORIZON GRID

Troubleshooting Guide

Version: 0.2.3
Documentation prepared: 2026-08-20

PREPARED FOR EVALUATION

Table of Contents

Troubleshooting Reference 3

Troubleshooting Reference

This chapter is a symptom-first reference: one entry per real problem, each with the same five fields -- Symptom, Cause, How to Check, Fix, Prevention. It complements, rather than repeats, the Health & Troubleshooting chapter (which covers the AI-unavailable, database-outage, cache-speed, provider-card, disconnected-investigation, export-limitation, and invalid-IOC scenarios from an analyst's point of view, with real screenshots) and the Final Release QA Report (the source for every "found and fixed" claim below). Where a scenario is already documented in full elsewhere, this entry gives the operator-facing summary and points back rather than re-explaining it end to end.

Every scenario below was either deliberately induced and observed, or organically encountered, during this platform's own QA passes -- none are hypothetical.

Frontend unavailable

Symptom: `http://localhost:3000` doesn't load, times out, or shows a browser connection-refused error.

Cause: Most commonly one of: `app-frontend-1` isn't running (crashed, never started, or Docker Desktop itself isn't running); the container is stuck at `Created` rather than actually started (a known bind-mount/anonymous-volume permission failure under Windows/WSL2 when running from a real dev tree rather than the production override); or the wrong port is being used because the install was configured with a non-default `HOST_PORT_FRONTEND`.

How to Check: Run the **Service Status** shortcut -- it reports `Web interface: OK/NOT RESPONDING` directly. If it reports not responding, `docker compose ps` (or the same info via **Diagnostics**) shows whether `app-frontend-1` is `Up`, `Created` (never actually started), or `Exited`.

Fix: If `Exited` or `Created`, `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d frontend` (what the **Restart Platform** shortcut does) recreates it. The production override (`docker-compose.prod.yml`) is what an installed copy already uses specifically because it strips the anonymous `node_modules` / `.next` volumes that fail to mount under an installed, non-git Program Files tree -- if you're troubleshooting a manually-run dev-tree Compose setup instead, dropping to the prod override is the fix, not a workaround. If the port is the issue, check `HOST_PORT_FRONTEND` in `.env` against what you're actually browsing to.

Prevention: Always use `docker compose -f docker-compose.yml -f docker-compose.prod.yml ...` (both files together) for anything other than active development against a live-editable source tree -- the dev-only base file's bind mounts are the entire reason this class of failure exists.

Backend unavailable

Symptom: `GET /health` doesn't respond; the frontend loads but every API call fails; Service Status reports `Backend API: NOT RESPONDING`.

Cause: Three real, distinct causes, in order of likelihood: (1) `alembic upgrade head` failed inside the backend container's own startup command (`sh -c "alembic upgrade head && uvicorn ..."`) -- migrations run synchronously *before* Uvicorn even execs, so a failed migration means no API start at all, not a stale-schema start; (2) Postgres wasn't healthy yet when the backend tried to start (the backend's `depends_on: postgres: condition: service_healthy` should prevent this, but a slow-starting Postgres on an under-resourced machine can still race it); (3) the container simply isn't running.

How to Check: `docker compose logs app-backend-1` -- structured JSON logs, so an Alembic traceback or a Postgres connection error is easy to pick out even in the same format as normal runtime logs. Confirm Postgres is healthy first (`docker compose ps` should

show `app-postgres-1` as `healthy`, not just `running`).

Fix: Once Postgres is confirmed healthy, `docker compose up -d backend` (not `restart` -- see the "a `.env` edit has no effect" pattern below, which is the same underlying mechanism) recreates the container and re-runs the migration attempt. If the migration itself is failing, the traceback in the logs names the actual revision/error -- that's a data-integrity question, not a container question, and should not be worked around by skipping the migration.

Prevention: Never run the built backend image directly with `docker run`, bypassing Compose -- the image's own baked-in `CMD` is plain `uvicorn app.main:app`, with no migration step; only the Compose `command:` override runs Alembic first.

Port already in use

Symptom: During installation or on `docker compose up`, a service fails to bind its port, or the Setup Wizard's port fields show a conflict.

Cause: Another process (or another instance of this platform, or a leftover container from a previous attempt) is already listening on the same host port -- most commonly 3000, 8000, 5433, 6379, 7475/7688, or 9200.

How to Check: The Setup Wizard checks this live, per field, as you configure ports -- entering a value the machine currently has bound shows "In use -- try `<next free port>`" right next to the field, computed by literally checking `Get-NetTCPConnection -LocalPort <port> -State Listen` on the host. Outside the wizard, the same check is a plain `netstat -ano | findstr :<port>` (or `Get-NetTCPConnection -LocalPort <port>`) to see what already holds it.

Fix: Either stop whatever else is using the port, or configure this install to use a different one -- the wizard's own suggested next-free-port (a linear scan upward, capped at 50 attempts) is a safe default. `Check-Prerequisites.ps1` (run automatically before installation) also checks free ports up front, but a conflict there produces a "Continue anyway?" prompt rather than a hard abort -- don't reflexively click through it without checking what's actually listening.

Prevention: If running multiple installs or environments on one machine (e.g. a dev tree and an installed copy simultaneously), give each a fully distinct set of host ports up front rather than relying on the conflict detection to catch it after the fact.

"API key not provided" / Test Connection always fails on an already-saved credential

Symptom: A provider or AI backend was configured and saved successfully (it shows "Configured" on the Manage Providers page), but clicking **Test Connection** on that same row reports an authentication failure -- as if no key were set at all.

Cause: This is real, and by design, not a bug: once a credential is saved, the platform never sends it back to the browser in a usable form -- the field shows only a masked preview (e.g. `sk-a****yz12`) as a `placeholder`, which is display-only. The credential input itself starts **empty** every time the row is expanded. `Test Connection` (`POST /api/v1/providers/{id}/test`) makes a real live HTTP call using exactly whatever value is literally typed into that field at the moment you click it -- it never falls back to the already-stored, encrypted value. Leaving the field blank and clicking Test Connection therefore tests an empty string against the real provider, which correctly comes back as an authentication failure, even though the real, saved credential is perfectly valid and is what every actual investigation will use.

How to Check: Look at the field itself -- if it shows greyed-out placeholder-style text rather than text you typed, it's empty, not populated with a hidden value. This is consistent for every provider that has a live Test Connection check (VirusTotal, AbuseIPDB, OTX, the abuse.ch-backed providers, NVD, Hybrid Analysis, Censys, PhishTank, Google Safe Browsing, urlscan.io).

Fix: Retype the real API key into the field before clicking Test Connection. This is only required for *testing* an already-saved credential again -- it is **not** required for the credential to keep working in real investigations; a saved, unmodified credential continues to be used correctly by every live lookup regardless of whether you ever re-test it.

Prevention: Treat "Configured" (the badge on the collapsed row) as the source of truth for "is a credential saved," and treat Test Connection as a check on whatever you're *about* to save, not a re-verification of what's already saved. This is a distinct, unrelated issue from the historical "Test Connection succeeds, but a real investigation still reports not configured" bug described in the Runtime Configuration Architecture chapter -- that one was a real freshness bug and has been fixed; this one is expected, current behavior arising from credentials never being round-tripped back to the browser in the clear.

Provider timeout / rate-limited

Symptom: One provider's card in an investigation shows "Timeout" or "Rate Limited" while others complete normally.

Cause: `Timeout` means the call exceeded `provider_timeout_seconds` (default 20s); `Rate Limited` means the vendor returned HTTP 429, 403, or 509 (509 is PhishTank's documented over-limit code). Only connection-level failures (`ConnectError` / `ReadTimeout` / `PoolTimeout`) are automatically retried, up to `provider_max_retries` additional attempts (default 2, exponential backoff) -- a 429/403/509 is reported immediately and is never retried, since retrying a rate limit immediately would make it worse.

How to Check: The provider's card in the investigation UI shows the status directly; `GET /api/v1/providers/health` shows whether this is an isolated blip (fine in the 7d/30d windows, degraded only in 1h) or a sustained pattern (degraded/down across all four windows -- worth checking whether a free-tier quota has been exhausted, or whether the vendor itself is having an outage).

Fix: Nothing to fix for an occasional rate limit -- it's expected behavior under real-world API quotas, and the investigation completes normally using every other provider's results regardless. For a sustained pattern, check the vendor's own status page and your account's quota/tier; a paid-tier key or a longer wait between bursts of lookups may be the only real fix if you're consistently hitting a free-tier ceiling.

Prevention: None of this is preventable at the platform level for a third-party vendor's own rate limit -- it's inherent to using free/low-tier external APIs. What the platform guarantees instead is that one provider's timeout or rate limit never blocks or delays the rest of the investigation, and never gets mislabeled as a different status (e.g. a rate limit never silently reads as "no data" or "clean").

AI unavailable

Symptom: The final assessment reads "Unknown," risk/confidence scores read 0, and a note explains AI summarization was unavailable.

Cause: The configured AI backend (Ollama by default) could not be reached, timed out, or was misconfigured. This exact scenario was deliberately created during this platform's own QA pass by making the AI backend unreachable, and every AI-dependent field correctly reported the failure rather than fabricating a plausible-looking result -- this is documented in full, with the exact real Ollama-unreachable and Ollama-timeout error strings, in the Health & Troubleshooting chapter and the Deployment and Troubleshooting chapter. Every provider's own raw data is completely unaffected -- only the AI-generated synthesis layer fails.

How to Check: The investigation's own assessment panel states the failure directly. `FinalAssessment.ai_outcome` for that investigation records `"failed"` (a genuine failure) rather than `"skipped_no_evidence"` (a correct decision not to call the AI because there was nothing to summarize) -- these are tracked as two different, honest outcomes, never conflated. For Ollama specifically, confirm it's actually running on the host (it's deliberately never containerized -- a large local model's mmap load was measured dramatically slower through Docker Desktop's WSL2 volume layer than direct host access).

Fix: Re-run the **Configuration** shortcut to confirm the AI backend setting, verify the model/service it points to is actually reachable from the host, then re-run the investigation -- the very next AI call after connectivity is restored succeeds normally, with no restart of the backend container required (AI backend resolution is a fresh database read and a freshly-built client on every single call, never a cached client).

Prevention: If running a local Ollama model, make sure the host has enough RAM/VRAM for the configured model and that Ollama itself is set to start automatically with Windows, so it's already warm before the first investigation of the day.

Linux-specific note: `host.docker.internal` (the address the platform uses to reach a host-native Ollama) is a Docker Desktop convenience that native Linux Docker Engine does not provide automatically. `docker-compose.yml` maps it explicitly (`extra_hosts: host.docker.internal:host-gateway` , supported since Docker Engine 20.10) so this resolves correctly on a real Linux install too — confirmed live on a fresh Ubuntu 24.04 install. If you're running an older Docker Engine that predates this feature, point `OLLAMA_BASE_URL` at the host's real LAN IP instead of `host.docker.internal` as a workaround.

Database unavailable

Symptom: The page shows a clear error, or an investigation won't load at all.

Cause: Postgres (`app-postgres-1`) is down or unreachable -- stopped, crashed, or still starting.

How to Check: `docker compose ps` shows whether `app-postgres-1` is `Up` / `healthy` ; `docker compose logs app-backend-1` shows the connection error the backend hit. This exact scenario -- the database deliberately stopped mid-request -- was tested during this platform's own QA pass: the result was a clear, fully logged error, never a silent failure, a crash, or corrupted data, and previously saved cases and Basket entries were confirmed completely intact once the database came back.

Fix: Restart the database (the **Restart Platform** shortcut, or `docker compose up -d postgres` directly), then retry. No data recovery step is needed for a clean stop/restart -- this is not a corruption scenario.

Prevention: Take regular backups anyway (§5 of the Operations Guide) -- a clean outage doesn't lose data, but it's not a substitute for having a real point-in-time backup for the scenarios that are actually destructive (disk failure, an accidental `docker compose down -v`).

Login failure

Symptom: Signing in returns an error instead of reaching the dashboard.

Cause: One of exactly two real, distinct backend responses: `401 Invalid email or password` (the email doesn't exist, or the password is wrong -- the message is deliberately identical either way, so a failed login attempt can't be used to enumerate valid accounts) or `403 Account disabled` (the account exists and the password may even be correct, but an administrator has disabled it).

How to Check: The error message itself distinguishes these two cases directly. If unsure whether an account exists at all, an administrator can check the Admin/user management page (ADMINISTRATION nav group, admin-only).

Fix: For `Invalid email or password` , double-check for typos, caps-lock, or whether the account was ever actually created -- the very first account ever registered on a fresh install automatically becomes the Administrator; every later registration becomes an Analyst by default. For `Account disabled` , only an administrator re-enabling the account resolves this -- there is no self-service path.

Prevention: Use the Setup Wizard's own re-run ("Configuration" shortcut) rather than trying to register a duplicate admin account if credentials are simply forgotten -- re-running the wizard on an existing install prompts for the existing admin email/password rather than creating a new one.

Permission denied

Symptom: A `403 Forbidden` response, with a message in the shape `Role '<role>' lacks permission '<permission>'` (e.g. `Role 'viewer' lacks permission 'lookup:create'`).

Cause: The signed-in user's role genuinely doesn't include the permission the attempted action requires. This is not a bug -- it's the RBAC system doing exactly what it's supposed to. The most common real case: a **Viewer** account (deliberately broad read-only access -- dashboard, provider health, lookups, evidence, cases, security assessments, all readable) attempting something Viewers are intentionally excluded from: creating an investigation, exporting results, running a security assessment, or touching provider/AI/user configuration (all of those require Analyst or Admin).

How to Check: The error message itself names both the role and the exact missing permission string, so there's no guessing which one is short. Compare against the three-role permission matrix (ADMIN has everything; ANALYST has lookup/evidence/analysis/hunting/copilot/basket/case/security-assessment/dashboard permissions; VIEWER has read-only lookup/evidence/case/security-assessment/dashboard access only) documented in full in the Security Architecture chapter.

Fix: Either have an administrator grant the user the Analyst or Admin role (if their actual job requires the action), or recognize that Viewer is working as intended and use an Analyst/Admin account for that specific action instead.

Prevention: Assign roles based on what a person actually needs to do, not defensively -- Viewer's read-only scope is deliberately generous (it includes the full dashboard and provider health, not just investigation results) specifically so that most "just looking" use cases never need Analyst access at all, which keeps the set of accounts that can create/export/configure things smaller and easier to audit.

Installer failure (fresh install crash-loops on a Postgres password error)

Symptom: Immediately after a fresh install finishes, the backend container crash-loops with a Postgres authentication error in its logs, even though the wizard just finished writing a brand-new `.env`.

Cause: A real, found-and-fixed bug, not a hypothetical. Postgres only ever applies the `POSTGRES_PASSWORD` environment variable to a **brand-new, empty** data volume -- once a volume has been initialized once, every later container start ignores that variable entirely and keeps whatever password is already baked into the volume. A fresh install with no existing `.env` always generates a new random database password. If a Postgres data volume from an earlier, abandoned install attempt on the same machine was never actually removed (e.g. `.env` was deleted, or an install was cancelled partway, but nothing ran `docker compose down -v` against it), that new random password can never match the old volume's real one -- and the backend crash-loops on an authentication error the instant it starts, on what looks to the person installing like a completely fresh, first-ever install.

How to Check: `docker compose logs app-backend-1` on the failing install shows a Postgres authentication error immediately, with no successful startup ever having occurred. `docker volume ls` showing a `postgres_data` volume already present before the very first successful start is the tell.

Fix (already applied): This has been fixed at the source. The Setup Wizard now detects, on a genuinely fresh install only (never on an upgrade/reconfigure of an existing install -- that path already correctly reuses its real existing password), whether a leftover Postgres volume from an earlier abandoned attempt exists -- identified by Docker Compose's own project/volume labels, not a hardcoded name -- and removes it before generating the new password, so the new password always initializes a genuinely empty volume. If you're hitting this exact symptom on a build that predates the fix, the manual equivalent is: `docker compose down -v` (removing volumes) before a clean reinstall, accepting that any data in that specific abandoned volume is unrecoverable anyway, since without a matching `.env` its password was never known to begin with.

Prevention: Let an install either fully complete or be fully removed (`docker compose down -v`) before starting another attempt on the same machine -- don't delete `.env` by hand as a way of "resetting" a partial install, since that's exactly the state that used to trigger this bug.

Invalid IOC input

Symptom: Typing a value into the investigation search returns `422 Could not determine IOC type; pass ioc_type_hint.` (surfaced in the UI as "Could not determine IOC type").

Cause: The value didn't match any of the IOC formats the platform's detector recognizes -- IPv4/IPv6, CIDR, domain, URL, file hash (MD5/SHA1/SHA256/SHA512), CVE/CAPEC/CWE ID, MITRE technique ID, ASN, TLS certificate fingerprint, YARA/Sigma rule text, and a handful of free-text categories (malware family, threat actor, campaign, file name). This is a deliberate, clean rejection, not a crash -- the detector runs a fixed sequence of pattern checks and simply returns "unknown" if none match, and the route layer turns that into an explicit 422 rather than guessing.

How to Check: Re-read the exact value for typos, stray whitespace, or a copy-paste artifact (a trailing period on a domain, a URL missing its scheme, a hash with the wrong character count for its claimed algorithm). If it's a legitimate IOC type the detector doesn't automatically recognize, the API accepts an explicit `ioc_type_hint` to bypass auto-detection entirely.

Fix: Correct the value and retry, or supply `ioc_type_hint` if calling the API directly rather than through the UI's guided input.

Prevention: None needed beyond normal input care -- this is working as designed, and matches the same principle used everywhere else in the platform: report an honest, specific rejection rather than silently guessing a type and producing a meaningless or misleading result.

Summary

Every entry above traces to a real, either deliberately-induced or organically-encountered condition, not a hypothetical -- the Postgres-password installer bug, the connection-pool/query-optimization dashboard failure, the NO_DATA-as-failure Provider Health bug, and the AI/database-outage scenarios were all found, fixed (where a fix was appropriate), and confirmed live during this platform's own QA passes, and are cited from the Final Release QA Report and the relevant architecture chapters rather than re-derived here. Two patterns recur across several entries and are worth remembering on their own: (1) the platform consistently prefers an honest, specific failure message over a silent guess or a fabricated success -- "Unknown," "Not Configured," "Could not determine IOC type," and the exact `Role '<x>' lacks permission '<y>'` string are all instances of the same design choice; (2) a failure in one subsystem (one provider, the AI backend, Celery) is consistently isolated from the rest of the platform rather than cascading -- a rate-limited provider doesn't block an investigation, a down AI backend doesn't touch provider data, and a stopped Celery worker doesn't affect a single live lookup.